

Conceptos avanzados de herencia

Índice

Antes de empezar	2
Herencia.....	2
Dynamic Binding	3
Polimorfismo	5
Contratos	6
Clases abstractas	7
Clases abstractas y métodos abstractos.....	9
Comportamiento abstracto e interfaces	10
Polimorfismo en Java.....	12

Antes de empezar

En caso de querer realizar pruebas con las porciones de código provistas en este apunte, recuerden que cada clase, cada clase abstracta, cada interface va en su propio archivo ".java". Asimismo, cada vez que hagan un método *main* para hacer alguna prueba, se recomienda hacerlo también en una clase separada.

Herencia

Vimos hasta aquí la principal ventaja de la herencia: reutilizar código. Se puede eliminar código duplicado "generalizando" o "abstrayendo" atributos y comportamiento en una clase más general, "clase padre" o "superclase". De esta manera, si hay que modificar el código, el cambio será en un solo lado y todas las subclases verán reflejada la modificación.

Mediante la herencia puedo garantizar que todas las clases "hijas" de una superclase tendrán todo el comportamiento, **no privado**, disponible. Es decir, al tener una superclase y subclases definimos "un protocolo común" para comunicarse (mediante mensajes, llamándole métodos) con la superclase y todas sus hijas. Como ya vimos, este protocolo se denomina "interfaz pública". Por ejemplo: una clase Animal establece un protocolo común, o **interfaz pública**, para todos los animales que yo defina (Perros, Gatos, Conejos... que **son un** Animal).

Según lo que vimos en un apunte anterior, todo comportamiento heredado puede "redefinirse", haciendo sobreescritura del método, reemplazando su comportamiento o alterándolo, reutilizando el heredado del padre. (Hay que recordar que esto no debe confundirse con la sobrecarga, que es un mecanismo mediante el cual podemos definir "variantes" de un método, manteniendo su nombre, pero alterando su firma).

Supongamos una empresa en la que hay Empleados, algunos rasos y otros Ejecutivos. Los empleados y ejecutivos realizan básicamente las mismas operaciones, solo que los ejecutivos tienen algunos beneficios. Por ejemplo, tienen un 50 % más de vacaciones que los empleados rasos. Además, tienen algunas operaciones extra, como cobrar un bono

anual y, también, cerca de retirarse puede elegir trabajar menos horas por día. Dado que tanto empleados como ejecutivos tienen datos en común, como nombre, DNI, edad, podemos a su vez poner una clase Persona por encima de Empleado y hacerlos disponibles a su clase “hija” Empleado y, por tanto, a su clase “nieta” Ejecutivo.

Dado que los objetos en una herencia se relacionan de forma de "ES UN" (a diferencia de TIENE UN como ocurre con la composición) es posible escribir el siguiente código:

```
Persona p = new Ejecutivo();
```

¿Por qué? Porque Ejecutivo ES UN Empleado y Ejecutivo ES UNA Persona, entonces, Ejecutivo ES UNA Persona. ¿Y puedo hacer al revés? Dado que no todas las personas llegan a ser ejecutivos, yo tengo que asegurar que x es un ejecutivo. Ya vimos en un apunte anterior el concepto de casting o casteo. Entonces:

```
Persona x = new Persona();
Persona y = new Ejecutivo()
Ejecutivo e = x; //no compila

// compila, pero no anda,
//dado que x no es una instancia de Ejecutivo
Ejecutivo e = (Ejecutivo)x;

Ejecutivo e = (Ejecutivo)y; //compila y funciona correctamente
```

Para asegurarme de que puedo "castear" puedo utilizar el operador "instanceof".

Dynamic Binding

El binding es un tema que escapa al alcance de la materia, pero es importante tener una noción muy básica para entender lo que se desarrollará en las siguientes secciones del apunte. Alcanza con mencionar que, en Java, como en muchos lenguajes, hay dos formas de “atar” una declaración con su tipo. El Static Binding ocurre en tiempo de compilación, es estático en el sentido que se analiza la atadura sin ejecutar el programa. Por otro lado, el Dynamic Binding se realiza durante la ejecución del programa. Por eso en algunos casos del casteo vimos que podemos “mentirle” al compilador, pero cuando se ejecuta el

programa puede fallar y, para eso, tenemos el instanceof. Si el instanceof se utilizar para analiza instancias, podemos afirmar que es algo que ocurre durante la ejecución.

Entonces, ¿que implica ejecutar la siguiente instrucción?

Perro p	=	new
		Perro();
(1)	(3)	(2)

(1) Se declara una referencia: es una variable. Todavía no se ha creado ningún objeto. Solo se indica a la JVM que reserve el lugar para una referencia que, eventualmente, apuntará un objeto Perro (Un objeto de tipo Perro).

(2) Se crea un objeto: se le indica a la JVM que reserve espacio para guardar un objeto Perro en la memoria.

(3) Asignación: se "ata" (bind) la referencia al objeto; hago que le referencia "apunte" a un objeto Perro.

Entonces, aquí, tanto la referencia como el objeto referenciado son del mismo tipo: Perro. Pero como vimos con personas y ejecutivos, es posible que la referencia y el objeto referenciado sean de distinto tipo. Ahora bien, como vimos, cualquier objeto que pase la prueba ES UN respecto de la referencia puede ser apuntado por ésta: (Ejecutivo ES UNA Persona, por eso Persona p = new Ejecutivo() es correcto). O sea, todo lo que extienda del tipo de la referencia, puede ser asignado a esta. Todo lo que extienda de lo que está en (1) puede aparecer en (2).

Lo interesante ocurre en tiempo de ejecución: solo cuando el programa corre, la JVM llamará al método del objeto referenciado.

Entonces, si hago:

```
Perro d = new Doberman();  
d.ladrar();//ladra como un doberman  
  
d = new Cocker();  
d.ladrar();//ladra como un cocker  
  
d = new Beagle();  
d.ladrar();//ladra como un beagle
```

Es decir que el perro d se puede comportar como un Doberman o se puede comportar como un Cocker o un Beagle. Es decir, que d toma varias formas.

Polimorfismo

Es la capacidad de un mismo objeto de comportarse como otro o la capacidad de un objeto de funcionar de diversas formas. En el caso anterior, un Perro se puede comportar como un Doberman, como un Cocker o como un Beagle porque estos SON Perros.

Sucede lo mismo si el objeto polimórfico es el recibido por un método como parámetro.

```
public class Nene {  
  
    molestarPerro(Perro p) {  
        System.out.println("Voy a molestar al perro");  
        System.out.println(p.ladrar());  
    }  
  
}
```

Ejecutando un main:

```
Nene n = new Nene();  
Perro d = new Doberman();  
Perro c = new Cocker();  
c.molestarPerro(d); //se escuchara el ladrido de un doberman  
c.molestarPerro(c); //se escuchara el ladrido de un cocker
```

Si usamos código polimórfico, podemos estar seguros de que modificaciones futuras, que agreguen nuevas subclases, no deberían afectar el código que ya se está utilizando ni su funcionamiento. Es decir, agregar subclases no "rompe" el diseño ni el código existente. Si

el código usa Perros (es decir, cualquier objeto que ES UN Perro) siempre que nuevas razas de Perros introducidas al sistema extiendan de Perro funcionarán correctamente.

Contratos

Con lo visto hasta ahora, para ver en acción el polimorfismo dependemos de “la buena conducta” del programador. Es decir, quien escriba el código sabe exactamente qué métodos debe pisar y cómo pisarlos, conoce sus firmas, cómo funcionan, etc. Dicho esto, si quien debe sobrescribir los métodos no conociera qué métodos pisar, sus firmas, etc., podría heredar de Perro, pero no necesariamente sobrescribir los métodos correctos. Quizás decida agregar algunos, quizá decida sobrecargar los heredados o quizá en un golpe de suerte justo pise los métodos correctos. En este caso no necesariamente podamos hacer uso del polimorfismo.

Una forma de dar al programador información sobre los métodos a sobrescribir para hacer uso del polimorfismo y así introducir nuevos perros al sistema sería mediante la documentación: se hace uso de javadoc o se entrega instrucciones específicas de cómo hacerlo. Aun así, no hay nada que obligue al programador a seguir esas reglas. Podemos incluir sanciones (¿descontarle algo de sueldo?) si no las cumple... pero aun así eso rompería el sistema, posiblemente en tiempo de ejecución.

Debería existir algún mecanismo que obligue al programador (que es el “usuario” de los métodos heredados) a sobrescribir ciertos métodos específicos, respetando sus firmas, sin depender de tantos factores externos para que nuestro sistema soporte el polimorfismo. Este mecanismo debería ser tan estricto que, si no se cumplen las reglas, el código ni siquiera debería compilar, mucho menos llegar a ejecutarse. Esta obligación será dada por los contratos.

Entonces, ¿cómo es posible garantizar que el polimorfismo funcione como funciona? En nuestras subclases respetaremos ese “protocolo común”. Es decir, si pisamos los métodos con sobreescritura, lo hacemos respetando la firma del o los métodos. Si respetamos la firma de los métodos, entonces respetamos el comportamiento tal cual está, respetamos

el "contrato" de un objeto. Entonces, el polimorfismo es posible solo respetando los contratos.

¿Cómo podemos asegurarnos de respetar los contratos? Podemos definir el comportamiento abstracto en una clase para asegurarnos de que cada subclase respete el contrato implementando "su manera" de llevar a cabo ese comportamiento. Es así como se podrá garantizar que las hijas de cierta clase realicen, sí o sí, ciertas acciones, solo que pueden hacerlo "a su manera". En Java definimos contratos mediante Clases Abstractas y mediante interfaces.

Clases abstractas

¿Qué ocurre si queremos definir este comportamiento abstracto? Una alternativa sería usar clases "abstractas". Con ellas podemos definir, o "declarar", SOLO comportamiento para asegurar un contrato entre clases y así hacer uso del polimorfismo. La intención de una clase abstracta es únicamente declarar comportamiento. De esta manera, la clase abstracta no se puede instanciar, solo las hijas de esa clase serán las que puedan instanciarse. Llamaremos a toda clase que no es abstracta clase concreta. Las clases abstractas dictan "qué hay que hacer", las clases hijas concretas dirán "cómo hay que hacerlo". De esta manera, podemos usar como referencia la clase abstracta, "atándole" una instancia de una clase concreta a esta y hacer uso efectivo del polimorfismo.

Definimos a las clases abstractas y el comportamiento en abstracto con la palabra clave "abstract". Cómo el comportamiento es abstracto (solo decimos qué hacer), los métodos abstractos no tienen código asociado, no tiene "cuerpo". Veamos un ejemplo:

```
public abstract class Perro {  
  
    public abstract String ladrar();  
  
}  
public Doberman extends Perro {  
  
}
```

En este caso, la clase Doberman arrojará un **error de compilación, porque no respeta el contrato** de la clase Perro. Es decir, si Perro dice qué se debe hacer, la hija Doberman, debe “explicar” cómo hacerlo. Llamaremos a esta operación “implementar” el método ladrar(). Si Doberman quiere SER UN Perro, entonces debe respetar el contrato de los Perros: debe implementar un método que se llame ladrar, que devuelva un String y que no reciba parámetros. En pocas palabras, debe sobrescribir todos los métodos abstractos definidos en Perro.

```
public Doberman extends Perro {  
  
    public String ladrar() {  
        return "ladro como dóberman";  
    }  
  
}  
  
public Labrador extends Perro {  
  
    public String ladrar() {  
        return "ladro como labrador";  
    }  
  
}
```

Cuando implementamos los métodos, estos dejan de ser abstractos, por eso en Doberman ya no usamos la palabra clave abstract. Las reglas para la implementación son las de la sobreescritura (de hecho, es lo que estamos haciendo, sobrescribiendo comportamiento abstracto) así que aplican las mismas reglas: respetar tipo, cantidad y orden de los parámetros. Si no lo hacemos, entonces no respetamos el contrato, si no respetamos el contrato la clase arrojará un error de compilación.

Una clase abstracta es una clase como cualquier otra y, por tanto, puede tener atributos y puede tener métodos concretos. Aun así, se debe tener en cuenta que solo los abstractos serán los que definan el contrato.

```
public abstract class Perro {  
    private String nombre;  
  
    public void setNombre(String nombre) {
```



```

        this.nombre = nombre;
    }
    public String getNombre() {
        return this.nombre;
    }

    public String ladrar();
}

```

¿Por qué tener métodos concretos en una clase que no se puede instanciar? Porque estos métodos son susceptibles de ser reutilizados.

Por otro lado, que una clase abstracta no se pueda instanciar, no significa que no pueda tener constructores, el objetivo es el mismo: puedo definir constructores para reutilizar código. Es importante recordar las reglas acerca de los constructores y su uso en herencia, junto con los posibles efectos secundarios de definir constructores adicionales al constructor por default.

Recordemos que las clases abstractas son como cualquier otra clase, en el sentido que las podemos usar como veníamos haciendo hasta ahora. Según lo que vimos hasta aquí:

```

Perro p = new Doberman();
System.out.println(p.ladrar());

p = new Labrador();
System.out.println(p.ladrar());

```

Clases abstractas y métodos abstractos

Vimos que las clases abstractas definen comportamiento abstracto mediante métodos abstractos. Si la clase tiene al menos un método abstracto, debe ser declarada como abstracta, PERO si la clase es abstracta no es necesario definir un método abstracto. A veces, interesa tener una clase abstracta con el solo hecho de contener métodos para ser reutilizados, pero no interesa tener instancias de esa clase, solo de las hijas.

A su vez, una clase abstracta puede definir algunos métodos abstractos y otros concretos, definiendo un comportamiento con "una implementación por default". La primera subclase concreta en la jerarquía deberá implementar todos los métodos abstractos. Sin

embargo, si una subclase es abstracta puede diferir la implementación de algunos e implementar otros, dejando a su primera subclase concreta la implementación de todos los abstractos que resten.

Comportamiento abstracto e interfaces

Problema

Definir comportamiento abstracto mediante clases abstractas es correcto, pero es importante notar que quita flexibilidad al dejar el comportamiento "atrapado" en una jerarquía. ¿Qué ocurriría si quisiera definir comportamiento abstracto, pero fuera de una jerarquía? ¿Qué pasaría si quisiera definir comportamiento que es tan abstracto, que no es parte de Perro, ni de animal, ni siquiera de SerVivo? Por ejemplo, vestir() (la gente le compra "ropa" a sus perros). Una solución sería incluir el método vestir() en la clase Perro, pero todas las subclases de perro podrían tener el comportamiento "vestir"... y no todos a los perros los visten... (todavía me falta ver un Gran Danés con un piloto para lluvia, o un Mastín Napolitano con un trajecito tejido a mano...).

Otra alternativa sería hacer el método vestir() abstracto e implementarlo en donde corresponda. Aun así, cuando llegue a la última clase concreta de la jerarquía me voy a ver obligado a implementar un método que a lo mejor un perro no debe llevar a cabo... Podría dejar implementación vacía. Pero no es lo que realmente corresponde, ¿o sí? Si una clase tiene una operación que no hace nada, no debería tenerla siquiera. Alternativamente, se podría colocar solo el método donde corresponda, pero se corre el riesgo de duplicar código sin mencionar que dependiendo de si se lo coloca en todos o en algunos, se pierde la ventaja del polimorfismo.

Para complicar las cosas un poco más, ¿qué ocurriría si tenemos DOS superclases? Una clase sería Perro y la otra Mascota, mascota podría tener el método vestir() y Perro los otros. Para evitar complejidades de la herencia múltiple, los creadores de Java hicieron que solo soporte la herencia simple.

Solución al problema. Las interfaces

Las interfaces son muy similares a las clases abstractas: se definen con la palabra clave "interface" en vez de "class". Todos sus métodos son abstractos, por lo cual no es necesaria la palabra "abstract" y, al igual que las clases abstractas, los métodos no definen un cuerpo. Para utilizar las interfaces se usa la palabra clave "implements" en lugar de "extends". Veamos un pequeño ejemplo:

```
public interface Perro {  
    String ladrar();  
}  
  
public Doberman implements Perro { //implements me va a  
obligar a cumplir el contrato  
    public String ladrar() {  
        return "ladro como dóberman";  
    }  
}
```

Y en un main:

```
Perro p = new Doberman();  
System.out.println(p.ladrar());
```

Los métodos de una interfaz son siempre públicos y abstractos, no es necesario poner public o abstract en la definición.

Lo que permiten las interfaces es independizarse de una jerarquía, permiten agregar comportamiento a una clase que no se obtenga desde un nivel superior en la jerarquía, se "enchufa" lateralmente a la jerarquía. Incluso podríamos hasta mezclar ambos mecanismos:

```
public Doberman extends Animal implements Perro {  
    public String ladrar() {  
        return "ladro como dóberman";  
    }  
}
```

Polimorfismo en Java

Según lo visto hasta aquí, las clases abstractas pueden definir un contrato y, por tanto, se utilizan para el polimorfismo. Las interfaces representan comportamiento puro, solo definen qué hacer, incluso sin depender de ninguna jerarquía.

Entonces, dado que en una clase abstracta podemos mezclar comportamiento abstracto con métodos concretos, no se puede garantizar la definición completa de un contrato. Es por eso que, en Java, solo hay polimorfismo si utilizamos interfaces.